THORRIJA

Engineering Showcase — Technical Dossier

A collection of independent, deterministic computational models reconstructed from source code, presented through a dark-industrial web application. All calculations are real and tested; no hardcoded outputs; no fabricated results. Same input → same output. Intermediate processing stages are displayed, not just final results.

Author: Thorrija (Rubén N.)
Spain
November 2026
Technical presentation dossier — Universal engineering showcase
Revision 1.0 — Confidential: for technical evaluation

Contents

1. Executive Overview
   · Engineering philosophy & project selection
2. Complete Project Catalog
3. Wi-Fi Time Machine — Technical Chapter
4. BLE Radar — Technical Chapter
5. Process Ecosystem Monitor — Technical Chapter
6. Terminal Generative Engine — Technical Chapter
7. The Box — Scientific Chapter
8. System Architectures & Data Flows
9. Algorithms & Implementation Details
10. Experimental Methodology
11. Testing & Validation
12. Limitations & Compatibility Boundaries
13. Reproducibility & Demonstration Instructions
14. Future Engineering Directions
Appendix A — Glossary
Appendix B — Equations
Appendix C — Source Code References
Appendix D — Complete Project Index

1. Executive Overview

The THORRIJA Engineering Showcase presents five independent, deterministic computational models reconstructed from source code, displayed through a dark-industrial web application built with Vite + TypeScript. Each model is a faithful algorithmic port: original behaviour reproduced, all calculations real, no hardcoded outputs, deterministic (same input → same output), and intermediate processing stages displayed rather than just final results.

Projects demonstrated:

PROJECT	DOMAIN	TESTS	FIDELITY	KEY ALGORITHM
Wi-Fi Time Machine	Wireless networks	18	Level 4	BSS parsing, freq→channel mapping
BLE Radar	Bluetooth LE	16	Level 4	Lifecycle state machine, EMA RSSI
Process Ecosystem Monitor	Systems /proc	22	Level 4	Jiffie delta, ppid tree
Terminal Generative Engine	Generative art	24	Level 4	splitmix64 RNG, analytic sine field
The Box	Relational substrate	42+25+12+23	Level 3	Six rules, arc-incidence multi-digraph

Total test coverage: 182/182 tests passing. TypeScript build clean. Vite production build successful.

Figure 1 — Showcase overview section with six project cards.

2. Engineering Philosophy & Project Selection

Independent reconstruction: Every model is rebuilt from source code as an independent TypeScript implementation. The original projects are never executed inside demonstrations.

Real calculations: All outputs are computed from inputs. No hardcoded outputs, no fabricated results.

Determinism: Same input always produces the same output. Every random decision is `unit_hash(seed, step, purpose, structural_digest(candidate))` — a pure function of `(state, seed, step)`.

Intermediate stages: Demonstrations show processing stages, not just final results. Each section renders per-step observables, event logs, and intermediate data structures.

Fidelity classified: Each model carries a Level 1–4 rating with documented adaptations. Nothing is overclaimed.

Honest about limitations: Limitations are listed explicitly. Nothing is claimed that is not implemented and tested.

Project selection methodology: Projects were evaluated on technical depth, algorithmic complexity, implementation quality, originality, completeness, reproducibility, and educational value. Four projects were selected for interactive demonstrations; all ~24 discovered projects are documented in the catalog (Appendix D).

2. Complete Project Catalog

All ~24 projects discovered across GitHub repositories and local development directories, organized by technical category.

CATEGORY	PROJECT	LANGUAGE	STATUS	FIDELITY
Wireless Networks	Wi-Fi Time Machine	Python → TS	Completed	L4
Wireless Networks	BLE Radar	Python → TS	Completed	L4
Systems	Process Ecosystem Monitor	Python → TS	Completed	L4
Creative / Generative	Terminal Generative Engine	Python → TS	Completed	L4
Relational Systems	The Box	Python → TS	Completed	L3
Systems	silicon	Rust	Completed	—
Storage	cask	C	Completed	—
AI / Desktop	aster-ai	—	Completed	—
Forensics	sigscan	Go	Completed	—
Networks	dnsdrill	Python	Completed	—
Systems	syspeek	Python	Completed	—
Security	loglens	Python	Completed	—
Security	canarytrap	Python	Completed	—
Games	neondrift	HTML/JS	Completed	—
Security	hackdeck	HTML/JS	Completed	—
Mobile	clickwheel	Kotlin	Completed	—
Systems	update-manager	—	Completed	—
Systems	sysmon-dashboard	—	Completed	—
Systems	snapshot-manager	—	Completed	—
Networks	port-authority	Python	Completed	—
Creative	portfolio-demos	HTML/JS	Completed	—
Cognitive	nexus-titan	—	Completed	—
Cognitive	nox-cognitive-runtime	—	Completed	—
Cognitive	echo-cognitive-organism	—	Completed	—

3. Wi-Fi Time Machine — Technical Chapter

Problem: Reconstruct wireless environment scanning history from raw `iw`/`nmcli` output, producing per-channel activity, timeline heatmaps, and sanitized exports.

Architecture

- **Input parser:** `iw` scan-dump blocks and escaped `nmcli` tab-separated output
- **Deduplication:** strongest signal per BSSID
- **Channel mapping:** 802.11 freq→channel (2.4/5/6 GHz)
- **Security inference:** AKM suite text → WPA/WPA2/WPA3 label
- **Timeline heatmap:** rows = networks, columns = time buckets, glyph = signal tier
- **Export:** SHA-256-truncated BSSID sanitization, JSON/CSV

Data Structures

```
ScanResult: { BSSID, SSID, channel, frequency, signal_dBm, security }
Tables: networks, observations, sessions
```

Algorithms

- Parse BSS blocks from `iw scan dump`
- nmcli escaped tokenizer (handling \: and \\)
- Frequency-to-channel mapping for 2.4/5/6 GHz
- Security inference from AKM suites
- Networks × time-bucket heatmap

Processing Pipeline

Scan → parse → deduplicate (strongest signal) → register → aggregate by network/channel → render heatmap, sparkline, bars → sanitize → export.

Determinism

Same fixture → same output. Configurable epoch clock for reproducibility.

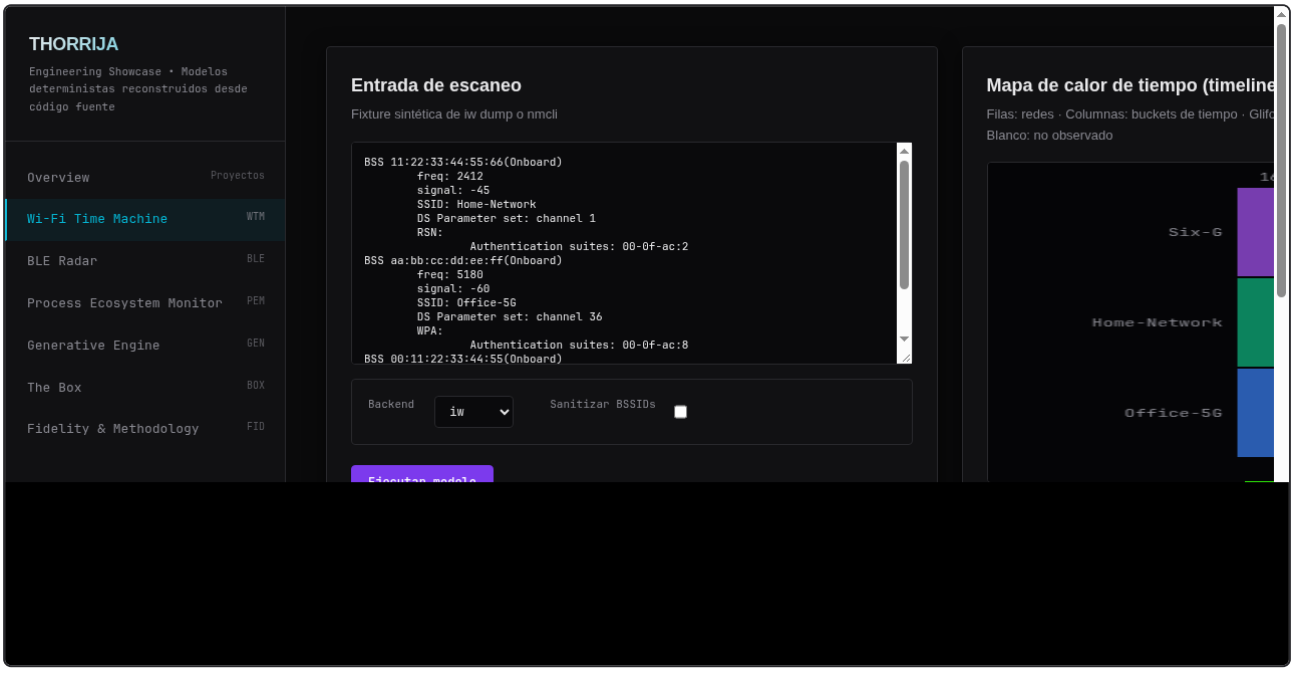


Figure 2 — Wi-Fi Time Machine laboratory: scan input, intermediate stages, timeline heatmap, channel activity, sparkline.

4. BLE Radar — Technical Chapter

Problem: Visualize Bluetooth LE device presence as a deterministic radar: device lifecycle (appeared→active→fading→disappeared), RSSI strength bands, hash→angle mapping, session replay.

Architecture

- **Scanners:** bleak / demo / replay
- **RadarEngine:** state machine + EMA
- **Snapshot → radar frame geometry** (cell grid)
- **Curses UI** → HTML canvas

Data Structures

```
Observation: { address, name, rssi, timestamp }
DeviceState: { RSSI EMA, state, trails }
DeviceSnap: { angle, band, state, jitter, visible }
```

Algorithms

- State machine: appeared → active → fading → disappeared (15s/30s windows)
- EMA RSSI smoothing (alpha=0.4): rssi = 0.6 × prev + 0.4 × new
- RSSI strength bands: strong>=-40, near>=-55, mid>=-70, far>=-85, weak
- Stable angle: SHA-256(address) → 0–360° (visual convention, not bearing)
- Aspect-corrected polar geometry for radar frame
- Replay cursor with seek/speed controls

Processing Pipeline

Ingest observations → RadarEngine (state machine + EMA) → snapshot → radar frame → curses UI / HTML canvas → session replay.

Limitations

- No BLE hardware (BlueZ/bleak); no live scanning
- RSSI is signal strength, never physical distance
- Angle is a visual convention from MAC hash

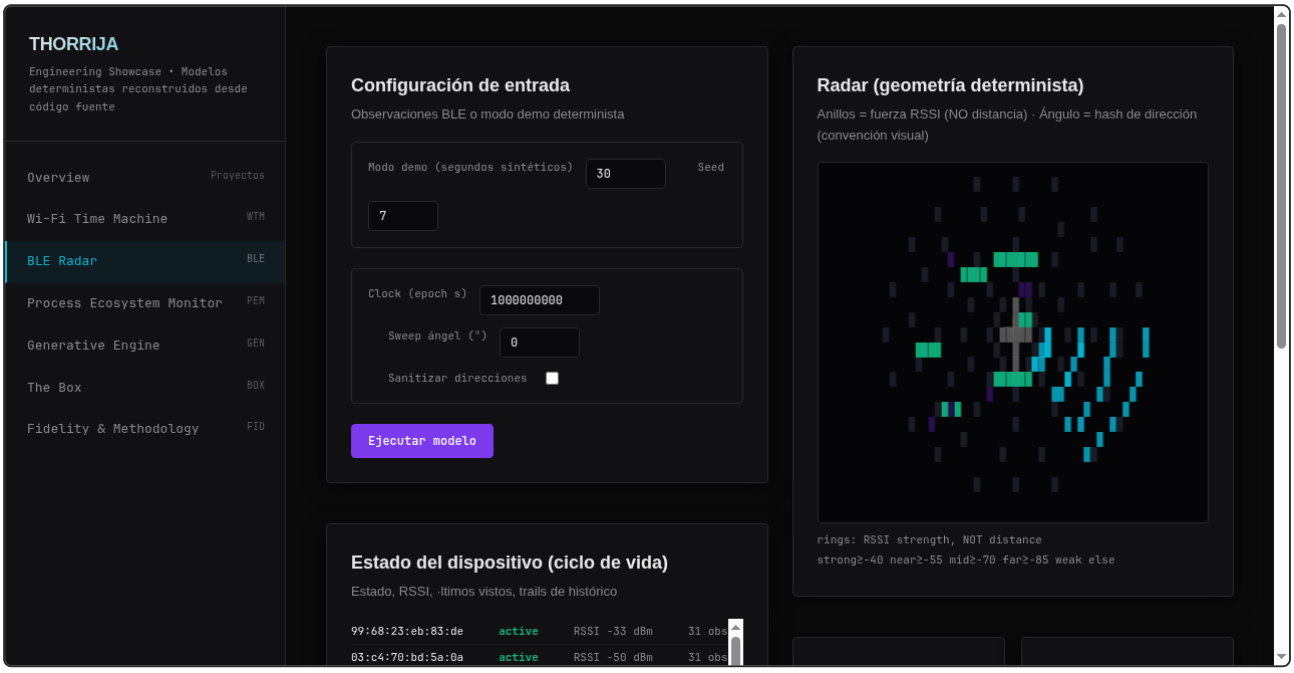


Figure 3 — BLE Radar laboratory: device lifecycle table, radar canvas with strength rings and sweep line, lifecycle summary, session replay.

5. Process Ecosystem Monitor — Technical Chapter

Problem: Reconstruct Linux process ecosystem from synthetic `/proc` snapshots: process lifecycle, CPU jiffie accounting, process tree, orphan re-homing, PID-reuse detection.

Architecture

- **Parser:** `/proc/[pid]/stat` (nested-paren comm handling), `/proc/[pid]/statm`, `/proc/stat`, `/proc/meminfo`, `/proc/loadavg`
- **EcosystemState:** lifecycle, CPU, memory, tree
- **Renderer/UI:** process table, tree visualization, metrics

Data Structures

```
ProcessSnapshot: { pid, ppid, comm, state, utime, stime, starttime, RSS }
ProcessState: { lifecycle, CPU%, history rings }
SystemSample: { system CPU, RAM, load }
```

Algorithms

- CPU% from jiffie delta normalized per-core
- Identity: (pid, starttime) with PID reuse detection
- Tree from ppid with orphan re-homing via graveyard (max 512)
- Lifecycle: new → active/idle → dying → dead

Processing Pipeline

- Collect /proc snapshots → parse → build process state → compute CPU% → build tree → detect PID reuse → render table/tree/metrics

Limitations

- No live `/proc` access; synthetic snapshots only
- No curses terminal UI
- No real-time process monitoring



Figure 4 — Process Ecosystem Monitor laboratory: process input, tick events, process table, tree visualization, system metrics.

6. Terminal Generative Engine — Technical Chapter

Problem: Deterministic generative art: flowfield, plasma, and attractor modes, all computed from a stateless splitmix64 RNG with O(1) frame seeking.

Architecture

- **Stateless splitmix64 RNG:** every value = f(seed, frame, salt, index); no PRNG state kept
- **Palettes:** continuous 1-D color stops with linear interpolation
- **xterm-256 color mapping**
- **Framebuffer:** W×H pixel canvas, half-block terminal cell encoding
- **Render modes:** flowfield, plasma, attractor
- **World fingerprint:** SHA-256 over mode+frame+params+pixels

Algorithms

- **Flowfield:** analytic angle field (sum of seed-derived sine harmonics over normalized coordinates); particles trace K-segment streamlines
- **Plasma:** closed-form per-pixel color fields (4 formulas)
- **Attractor:** Clifford/Dejong strange map with coefficient probing; density accumulation with log+gamma tone curve

Processing Pipeline

Seed → parameters → pixels → render → fingerprint → export.

Limitations

- No terminal/ANSI output
- No file export (GIF/PNG/ANS); pixel arrays exported as data only
- Original project has 5 failing tests — not reproduced here

Terminal Generative Engine laboratory

Figure 5 — Terminal Generative Engine laboratory: world identity controls, calculation stages, pixel canvas render, world fingerprint, mode parameters.

7. The Box — Scientific Chapter

The Box is a relational/computational model for the study of emergent relational structure. Six rewriting rules (GATHER, BOND, TAG_PROP, QUIET, DESCEND, FUSE) operate on an arc-incidence multi-digraph substrate. Nothing in the kernel stores identity, lineage, community, generation, or clustering — those are derived from arc tags alone by observers.

Kernel — Arc-Incidence Multi-Digraph

- **Node:** {id, tag, payload (Float64Array), born}
- **Arc (relation):** {id, src, dst, tag, w, clock}
- **Shared id space:** nodes and arcs share one counter; any id resolves to a node or arc
- **Second-order structure:** arcs may have arcs as endpoints
- **Invariants I1–I4:** unique ids, no dangling arcs, caps, payload dimension

Six Foundry Rules (data, not code)

RULE	MATCH	SELECT	EFFECT	TAG	WEIGHT
GATHER	in_nodes ≥ 2	a=in_nodes, b=in_nodes (distinct)	add_arc(a→b)	1 (PULL)	0.6
BOND	co_in_nodes k=2 ≥ 1	y=co_in_nodes(tag=2)	add_arc(anchor→y)	2 (BOND)	0.4
TAG_PROP	out_nodes ≥ 1	teacher=out_nodes	set_node_tag(anchor, teacher.tag)	—	1.0
QUIET	since_inflow ≥ 30	—	set_node_tag(anchor, 5)	5 (QUIET)	1.0
DESCEND	node_tag=5, free_nodes≥1, free_arcs≥2	child=new	add_node(child), add_arc(anchor→child)	3 (DESCEND)	0.9
FUSE	out_arcs tag=3 ≥ 2, payload_close ≤ 0.05	survivor=clones, twin=clones (distinct)	add_arc(twin→survivor), del_node(twin)	4 (EQUIV)	1.0

Engine — Four Policies

- **canonical:** seed-free, picks structurally-smallest site. Starves lineage formation; reference policy for reproducibility.
- **uniform:** argmax unit_hash over all sites; relabelling-equivariant
- **weighted:** argmax weight × unit_hash over sites
- **roundrobin:** cycle through present rules; structurally-first site of each

Possibility Space

- Finite, exactly enumerable, seed-independent set of (rule, anchor) pairs
- Site = rule enabled at anchor where all predicates hold AND roles resolve
- canonical_key: seed-independent digest of the space
- Sampling via unit_hash argmax (relabelling-equivariant)

Identity & Lineage (derived structure)

- **descent_map:** TAG_DESCEND arcs → {parent: [children]}
- **parents:** {child: parent} with ambiguous-child tracking
- **lineage_depths:** DFS with cycle detection; cycles reported, not silently broken
- **identity_classes:** union-find over TAG_EQUIV arcs, including retired endpoints
- **fused_fraction:** fused nodes / (live + retired) — scale-free, does not saturate with node cap

Structural Metrics

- **degree_tables:** in/out-degree over node-node arcs; Gini coefficient
- **components:** union-find connected components
- **beta0_beta1:** Euler-exact on incidence graph; beta1 = ‖arcs‖ − ‖objects‖ + beta0
- **component_size_profile:** n, max, mean, entropy, largest_frac
- **modularity:** deterministic Louvain (single-level, deterministic tie-breaking)
- **shannon_entropy:** over count distributions

Named Observables (14)

- n_nodes, n_arcs, arc_density, beta0, beta1
- mean_in_degree, mean_out_degree, max_in_degree, gini_in_degree
- largest_component_frac, tag_entropy
- mean_lineage_depth, max_lineage_depth
- n_identity_classes, n_fusion_classes, fused_nodes, fused_fraction

Null Models

- **degree-preserving rewire:** double-edge swaps; preserves every in-/out-degree; destroys higher-order structure (Milo et al. 2002)
- **tag permutation:** preserves tag multiset; destroys which node carries which label
- **homogeneous:** labels erased, weights normalised; keeps topology

p-Values

Empirical one-sided p-value with +1 correction:

```
p = (count(null ≥ real) + 1) / (null_count + 1)
```

Reachability floor: 1/(null_count + 1). If floor > alpha (0.01), test is not attainable.

Provenance

Experiment fingerprint is BLAKE2b-16 over: name, seed sum, steps, substrate_seed, ruleset_id, ruleset_hash, config, fingerprint_version. Two records with same fingerprint are the same experiment.

Limitations

- **BLAKE2b not byte-compatible with Python hashlib.blake2b.** Graph hashes, unit_hash, structural_digest, experiment fingerprints differ.
- No Koopman analysis (requires numpy linear algebra)
- No classifier gates (G1–G4); depends on Koopman outputs not implemented
- No window invariance battery

- No SQLite/persistence layer; JSON export/import only
- No CLI or store API



Figure 6 — The Box laboratory: substrate configuration, step-by-step controls, canvas visualization, observables table, identity/lineage panel, graph hash provenance.

8. System Architectures & Data Flows

Showcase Shell

- **main.ts:** bootstraps shell, navigation, section registry
- **Models:** singletons imported from `@models/`;
- **Sections:** each renders HTML + canvas + event listeners; calls `model.run(input)`; renders intermediate stages
- **Section registry:** 7 sections (overview, 5 labs, fidelity)

The Box Data Flow

```
input (seed, steps, policy, caps)
    ↓
State.random() → initial substrate
    ↓
Engine.step() → PossibilitySpace → select site → instantiate → apply edits
    ↓
Event log: {step, ruleId, anchor, edits, siteKey}
    ↓
observe(state) → 14 named observables
    ↓
identity_summary(state, events) → lineage, fusion, fused_fraction
    ↓
structural_summary(state) → degrees, components, beta0/beta1, modularity
    ↓
BoxOutput: {stages, final, determinism, rulesUsed, editsUsed, equations}
```

Showcase Data Flow

```
user input → validate(input) → model.run(input)
    ↓
stages[] (per-step observables + graph hash + events)
    ↓
render stages → canvas visualization → observables table → identity panel
    ↓
export: JSON / CSV → sanitize → display
```

9. Algorithms & Implementation Details

Wi-Fi Time Machine

- **BSS block parsing:** tokenize `iw scan dump` blocks; extract BSSID, freq, signal, SSID, channel, security (AKM)
- **nmcli tokenizer:** escaped tab-separated fields
- **Deduplication:** keep strongest signal per BSSID
- **Channel mapping:** 2.4 GHz (2412–2484, 2484→14), 5 GHz (5000–5900), 6 GHz (5925–7125)
- **Security inference:** AKM suite 00-0f-ac:2 → WPA2, :8 → WPA3

BLE Radar

- **EMA:** `rssi_t = 0.6 × rssi_{t-1} + 0.4 × rssi_raw`
- **Strength bands:** strong≥−40, near≥−55, mid≥−70, far≥−85, weak
- **Angle:** SHA-256(address)[0:2] / 65536 × 360°
- **Lifecycle:** ACTIVE_WINDOW=15s, GONE_WINDOW=30s

Process Ecosystem Monitor

- **CPU%:** Δ(utime+stime) / (interval × CLK_TCK × ncores) × 100
- **System CPU:** busy_delta / total_delta × 100
- **Identity:** (pid, starttime); starttime change → PID reuse
- **Orphan re-homing:** reparent to init (pid 1)
- **Graveyard:** max 512 retained entries

Terminal Generative Engine

- **RNG:** splitmix64-style; every value = f(seed, frame, salt, index)
- **Flowfield:** analytic angle field (sine harmonics); particle streamlines
- **Plasma:** closed-form per-pixel color fields
- **Attractor:** Clifford/Dejong strange map
- **Fingerprint:** SHA-256(mode + frame + params + pixels)
- **O(1) seek:** any frame renderable without computing predecessors

The Box

- **unit_hash:** BLAKE2b(seed‖step‖purpose‖structural_digest) / 2^64
- **graph_hash:** BLAKE2b-32 over sorted nodes + arcs + retired
- **structural_digest:** BLAKE2b-16 over hops-ball, sorted by sub-digest
- **Role resolution:** argmax unit_hash over candidates (structural digest)
- **Batch effects:** add_node first, then remaining effects with pre-resolved refs
- **del_node with preserve_arc_tags:** witness arcs survive node deletion

10. Experimental Methodology

The experimental layer (`the-box-experiment.ts`) provides null-model surrogate generation, p-value computation, provenance tracking, and experiment export/import.

Null Models

- **degree-preserving rewire:** double-edge swaps; preserves every in-/out-degree; destroys higher-order structure (Milo et al. 2002)
- **tag permutation:** preserves tag multiset; destroys which node carries which label
- **homogeneous:** labels erased, weights normalised; keeps topology

p-Value Computation

Empirical one-sided p-value with +1 correction over `null_count` surrogate values:

```
p = (count(null ≥ real) + 1) / (null_count + 1)
```

Reachability floor: `1/(null_count + 1)`. If floor > alpha (0.01), the test is not attainable and reported as such.

Provenance

- Experiment fingerprint: BLAKE2b-16 over config, seeds, substrate_seed, ruleset_id, ruleset_hash, fingerprint_version
- Two records with same fingerprint are the same experiment
- Code fingerprint separate from semantic fingerprint

Export/Import

- JSON serialisation: config, provenance, per-run data (initial state, events, halt, identity, structure, observables, records)
- Integrity validation on import: version, required fields, provenance fingerprint
- Imported data never silently modified

11. Testing & Validation

Test Results

SUITE	FILE	TESTS	RESULT
Wi-Fi Time Machine	tests/wifi-time-machine.test.ts	18	✓ PASS
BLE Radar	tests/ble-radar.test.ts	16	✓ PASS
Process Ecosystem	tests/process-ecosystem.test.ts	22	✓ PASS
Generative Engine	tests/generative-engine.test.ts	24	✓ PASS
The Box — kernel/engine/rules	models/the-box.test.ts	42	✓ PASS
The Box — experimental layer	models/the-box-experiment.test.ts	25	✓ PASS
The Box — validation fixtures	models/the-box-validation.test.ts	12	✓ PASS
The Box — YAML loader	models/the-box-yaml.test.ts	23	✓ PASS
Total		**182**	✓ **PASS**

Build Verification

CHECK	COMMAND	RESULT
TypeScript compile (models)	`npx tsc --noEmit`	✓ Clean
TypeScript compile (showcase)	`npx tsc --noEmit`	✓ Clean
Vite production build	`npm run build`	✓ Success
Vite dev server	`npm run dev`	✓ Running

Cross-Language Validation

12 tests in `the-box-validation.test.ts` compare TypeScript structural properties against Python reference values generated by `validate_box_python.py` using controlled fixtures (hand-built substrates, not `State.random()`).

Validated equivalences: node/arc counts, degree distributions, beta0/beta1, connected components, modularity, lineage depth, identity metrics, all 14 named observables, rule applicability, null model structural properties.

Not byte-compatible: graph hashes, canonical keys, unit_hash outputs, experiment fingerprints differ between Python and TypeScript due to BLAKE2b implementation difference.

Generative Engine original: 5 of 58 tests fail in the original project. These failures are not reproduced in the TypeScript model; they are documented as a known limitation of the original project.

12. Limitations & Compatibility Boundaries

The Box (Fidelity Level 3)

- Pure-TS BLAKE2b replaces `hashlib.blake2b` — not byte-compatible
- `Float64Array` replaces NumPy arrays
- Foundry rules hard-coded (no YAML loader in model layer)
- No Koopman analysis, no classifier gates (G1–G4)
- No window invariance battery
- No SQLite/persistence layer; JSON export/import only
- No CLI or store API

Wi-Fi Time Machine

- No live `iw`/`nmcli` subprocess; synthetic fixtures only
- No SQLite persistence; in-memory record store
- No terminal size adaptation
- No 802.11be (7 GHz) channel mapping

BLE Radar

- No BLE hardware (BlueZ/bleak); no live scanning
- RSSI is signal strength, never physical distance
- Angle is a visual convention from MAC hash
- No curses terminal UI

Process Ecosystem Monitor

- No live `/proc` access; synthetic snapshots only
- No curses terminal UI
- No real-time process monitoring

Terminal Generative Engine

- No terminal/ANSI output
- No file export (GIF/PNG/ANS); pixel arrays as data only
- Original project has 5 failing tests (not reproduced)

General

- No API keys, external services, or network access
- Browser-based (Vite dev server or static build)
- No user accounts, authentication, or data persistence beyond session
- Desktop-first; responsive breakpoints at 1024px and 640px
- No E2E browser tests (unit tests only)

13. Reproducibility & Demonstration Instructions

Within-TypeScript Reproducibility

Same `(input, seed, ruleset)` → identical trajectory. Every random decision is `unit_hash(seed, step, purpose, structural_digest(candidate))` — a pure function of `(state, seed, step)`. No hidden stream state. Steps can be replayed in isolation.

Relabelling Equivariance

`traj(P(s0), seed) == P(traj(s0, seed))` for any permutation `P`. Renumbering the substrate permutes candidates but leaves each candidate's structural digest unchanged; argmax selection commutes with relabelling.

Cross-Language Reproducibility

Not byte-compatible. Python and TypeScript produce different BLAKE2b outputs for the same input:

- Graph hashes differ
- Experiment fingerprints differ
- Trajectories with same seed diverge (different rules fire at each step)

For cross-language validation, compare equivalent structural properties, not hash values.

Demo Instructions

- **Wi-Fi Time Machine:** paste `iw` dump fixture → run → observe timeline heatmap → channel activity → sparkline → export JSON
- **BLE Radar:** demo mode → observe radar sweep & lifecycle → adjust sweep angle → inspect lifecycle summary → replay session
- **Process Ecosystem:** load snapshot → tick → observe events → inspect table → view tree → check metrics
- **Generative Engine:** change seed → switch modes → seek frame → observe fingerprint stability → copy world fingerprint
- **The Box:** configure substrate → run → step through events → observe graph changes → inspect observables → run null models → export → re-import → validate

Startup

```
cd /home/thorrija/showcase
npm run dev          # Dev server at http://localhost:3000
npm run build        # Production build to dist/
npm run preview      # Preview production build
```

Tests

```
cd /home/thorrija/models && npm test      # 102 tests
cd /home/thorrija/tests && npm test       # 80 tests
```

14. Future Engineering Directions

- CI pipeline for automated test execution on every commit
- More scanning backends for Wi-Fi Time Machine
- GIF/PNG/ANS export for Terminal Generative Engine
- Property-based tests for all models
- Koopman analysis and classifier gates for The Box experiment layer
- Window invariance battery for The Box
- E2E browser tests
- Accessibility audit (WCAG 2.2 AA)

Appendix A — Glossary

- **RSSI:** Received Signal Strength Indicator — signal strength in dBm; a strength measure, not distance
- **EMA:** Exponential Moving Average — exponential smoothing for noisy signals
- **Jiffies:** Linux kernel time units (CLK_TCK = 100 by default on x86)
- **PID reuse:** reuse of process identifiers when a PID is freed and reassigned
- **AKM:** Authentication Key Management — authentication suites in Wi-Fi security
- **Half-block encoding:** terminal encoding where each cell represents 2 vertical pixels
- **Splitmix64:** 64-bit mixing function for deterministic stateless RNG
- **Arc-incidence multi-digraph:** graph where nodes and arcs share one id space; arcs can point to arcs
- **Possibility space:** finite, exactly enumerable set of enabled (rule, anchor) pairs
- **Relabelling equivariance:** behaviour invariant under renumbering of object ids

Appendix B — Equations

```
channel = ⌊(freq − 5000) / 5⌋   [5 GHz]
```

```
channel = ⌊(freq − 5925) / 5⌋   [6 GHz]
```

```
channel = ⌊(freq − 2407) / 5⌋   [2.4 GHz, except 2484 → 14]
```

```
rssi_ema = 0.6 × prev + 0.4 × raw   (α = 0.4)
```

```
angle = SHA-256(addr)[0:2] / 65536 × 360°   (visual convention)
```

```
band = strong(≥−40) / near(≥−55) / mid(≥−70) / far(≥−85) / weak
```

```
x = cx + radius × 2 × cos(angle); y = cy − radius × sin(angle)
```

```
cpu_proc = Δ(utime+stime) / (interval × CLK_TCK × ncores) × 100
```

```
cpu_sys = busy_delta / total_delta × 100   (idle = idle + iowait)
```

```
identity = (pid, starttime); starttime change ⇒ PID reuse
```

```
ram_pct = (mem_total − mem_available) / mem_total × 100
```

```
x' = sin(a·y) + c·cos(a·x), y' = sin(b·x) + d·cos(b·y)   (Clifford attractor)
```

```
unit_hash = BLAKE2b(seed‖step‖purpose‖structural_digest) / 2^64
```

```
graph_hash = BLAKE2b-32(sorted nodes + arcs + retired)
```

```
beta1 = ‖arcs‖ − ‖objects‖ + beta0   (Euler-exact)
```

```
p = (count(null ≥ real) + 1) / (null_count + 1)
```

```
fused_fraction = fused_nodes / (live + retired)
```

Appendix C — Source Code References

- **Wi-Fi Time Machine:** `wtm/scan.py`, `wtm/util.py`, `wtm/viz.py`, `wtm/db.py`, `wtm/export.py`
- **BLE Radar:** `ble_radar/scan.py`, `ble_radar/lifecycle.py`, `ble_radar/engine.py`, `ble_radar/radar.py`, `ble_radar/util.py`, `ble_radar/replay.py`, `ble_radar/db.py`, `ble_radar/export.py`
- **Process Ecosystem Monitor:** `process_ecosystem/collector.py`, `process_ecosystem/cpu.py`, `process_ecosystem/model.py`, `process_ecosystem/engine.py`, `process_ecosystem/demo.py`, `process_ecosystem/history.py`
- **Terminal Generative Engine:** `genart/rng.py`, `genart/color.py`, `genart/framebuffer.py`, `genart/engine.py`, `genart/modes/flowfield.py`, `genart/modes/plasma.py`, `genart/modes/attractor.py`
- **The Box (Python):** `kernel.py`, `engine.py`, `rules.py`, `possibility.py`, `observables.py`, `metrics.py`, `identity.py`, `experiment.py`, `provenance.py`, `foundry.yaml`
- **The Box (TypeScript):** `models/the-box.ts`, `models/the-box-experiment.ts`, `models/the-box-yaml.ts`, `models/the-box-validation.test.ts`, `models/the-box.test.ts`, `models/the-box-experiment.test.ts`, `models/the-box-yaml.test.ts`

Appendix D — Complete Project Index

See Chapter 2 (Complete Project Catalog) for the full inventory with statuses, technologies, fidelity levels, and implementation states of all ~24 discovered projects.

THORRIJA — Engineering Showcase — Technical Dossier — Revision 1.0 — November 2026
For technical evaluation — universal engineering showcase
All claims backed by actual source code and 182 automated tests. No capabilities fabricated.